SENG202 – Software Engineering Project Workshop

Project Description – Clean Community City

Context

New Zealand prides itself on its beautiful natural landscapes and clean environment. A nice viewpoint is always nearby, and cities also offer lots of amenities, places to explore and things to discover. Knowing where to find all the best sights can be difficult for those new to the city or just exploring for a few days while on holiday. For those in town all year long being able to report issues or share rating/reviews helps give back to the community.

Introduction

The purpose of this project is to support the community in caring for their city and encouraging responsible use of public spaces.

Initial design considerations

- Your system must be implemented in Java as a desktop application (we'll be using JavaFX and possibly other libraries). Please be careful to evaluate external libraries before committing to use them as they may over-complicate implementation or come with restrictive licensing requirements.
- Your app and deliverables must be buildable and runnable on the lab machines in the CSSE computer labs (including the operating system, version of Java (currently 25), etc.).
- The application should be designed as a single-instance application, i.e. one user would use the app at a time however different users may make use of it at different times.
- Your app should perform efficiently with realistic data sets, such as any provided to you. However, you are likely to want to use smaller/simpler data sets during development.

Non-functional Requirements

Your app will deliver features as described in the functional requirements you will develop. However, your app is also expected to meet a number of non-functional requirements (NFRs) — these are properties of the overall software system, and should be met at all stages of development. Here are NFRs (sometimes referred to as Quality Requirements) your application must satisfy:

1. **User Assistance**. UI elements have tooltips to indicate the functionality they provide.

2. **Recognizability (UI consistency)**. This includes things like button placement in dialogs (e.g. “Cancel” always left of “OK”), defaults (e.g. “OK” always the default in dialogs) and font & colour schemes.
3. **Time performance**. Operations should normally complete within a few seconds. Users should see some indication (wait cursor, progress bar, ...) that the program is working if appropriate.

   This article by Jakob Nielsen has some relevant ideas, related to the three quality requirements hereabove.
4. **Robustness (User errors protection and auditability)**. Errors should be detected and handled consistently. The user should never see an error traceback. Logging is likely to be valuable in order to identify and resolve issues (e.g. malformed or missing data items).
5. **Learnability (user guide/tutorial)**. Well-structured documentation to support new users should be provided. This may include a range of media (images, sample data sets, video tutorials, ...) and the Goldilocks principle should be applied. User documentation is most important for the final deliverable when the application is stable.
6. **Licences**. The application complies with the licences of any included libraries and does not impose financial or other detrimental conditions on its users.
7. **Inclusivity and Biculturalism**. The application recognizes and incorporates te reo Māori where it is provided in all data points and location names.
8. **Responsibility**. Your application aims at providing genuinely useful services and should not lead to detrimental or dangerous behaviours, nor contain misleading information.
9. **Security**. The application appropriately handles sensitive user data - including irreversibly hashing user passwords, managing access rights and privacy, and not directly exposing keys to third party services.

Overview of features

The precise details of your application’s functionality and interface are up to you. In order to provide some initial guidance, we have grouped the basic key features of the application into core features and divergent features listed below.

Core Features (Feature Packages)

Packages A, B, & C are mandatory and should be implemented as a starting point for your application. The highlight packages detailed below give different possible spins you can take on the idea, picking one or two of these early on will help guide the exact details for the basic feature packages. For example, deciding to focus on tourism will lead to using different underlying data compared to asset management.

It is unlikely that your team will have time to develop all the features listed into detailed designs and to implement them subsequently. You will need to define feature ordering and precise their scope. Furthermore, the features listed in the following sections may lack detail, may be contradictory, may differ in business value and may require widely different amounts of effort to implement. You, in collaboration with your team and the teaching team, will refine the initial features and express the details in terms of UML diagrams, acceptance tests and other relevant documents.

Ultimately, the features of your application, and the order in which they will be implemented, **are decisions for your team to make.** These, in turn, will be influenced by the stakeholders you have identified, the system context you have identified, and other factors. We encourage you to **discuss your plans and challenges with the teaching team and other students** at our weekly meetings.

Feature package A (Basic UI and Loading Data)

The GUI should include a way for users to browse the underlying data, this may simply include reading data from individual supporting files at runtime.

- The main GUI of the application is split into several parts (panels) and has a menu bar (and possibly also a tool bar).
- The application should be able to load existing data from external sources (see below). Datasets may contain many records and therefore it may not always be practicable or convenient to view them in full. Therefore, for initial development and early testing, you may wish to use a subset of that data. You can open the files in an ASCII text editor, or other suitable component, to view and search the data.
- You may be using data sets which have differing schemas so may need to allow for additional/missing attributes in some cases.
- The application can show the content of the external data file(s) in a table ("raw data viewer"). All relevant data fields should be shown for a record. Column headers should be displayed if they are available.

Feature package B (Persistent Data Storage)

A complex application should have support for persistent data storage and handle its data in an efficient way.

- Data and any additionally calculated information must be stored persistently. Users should be able to access this information when closing and re-opening the application. This requires a data structure and storing mechanism. You will need to define your own data format for storing data in your application. This data format will most likely overlap with the data format of the external data. Also, you need to think about how different lists of data are stored.

- Users should be able to persistently store newly added records and also calculated data.

When implementing persistent storage, we suggest an iterative approach. You could start by defining a data access layer (DAL) with high-level methods (e.g., getAllRecords(), getAllRecordsInDateRange(), etc.) and then implement the actual persistence in a very simple way to start with (e.g., [text] files). Once you get a bit further into COSC265 you could then easily modify the functionality underneath the DAL to use an SQL database (e.g., SQLite) without having to modify much of the rest of the application.

Feature package C (Map Browsing)

When dealing with geospatial data including interactive visualizations of data on a map is an important way to better communicate the data and relation between points.

- Users should be able to view data points, clusters, or regions on a map.
- Users should be able to pan and zoom the map to view different locations in more detail. You may wish to restrict the mapped region such that it prioritises your data.
- Basic data points (i.e. pins/markers) should be interactable (i.e. clickable) and show the user more information. You may also consider using custom markers for different data point types

Large datasets can lead to performance challenges when dealing with maps, you may wish to consider grouping points or allowing filtering of datapoints to reduce the number being calculated and displayed at once

Feature package D (Publishing and Updating data)

Showing data exported from open data sets (and other sources) is a great start but the data is restricted by its age and correctness. Allowing users to add new data points or update existing ones allows for more reactive and correct information.

- Users should be able to add new data points (for relevant types, i.e. you may allow a user to add a point but not a more complex path or region).
- You may consider allowing adding new data points easily by clicking locations on the map or entering an address.
- There may be other types of data that are simply entered as forms, these may optionally relate to other existing points.
- Users should be able to update information about existing data points when viewing them. You may wish to restrict exactly what fields can be edited.

Feature package E (Graphs & Charts)

Users can gain much useful information from the numeric and text values in the data. However, the number of fields and records can lead to information overload — the user

can be overwhelmed by the sheer volume of data. As expressed in the adage “a picture is worth a thousand words”, it can be more effective to visualise data graphically. Information visualisation is a discipline in its own right, but you are likely to be familiar with a number of basic 2-dimensional graphical views of data. Histograms, bar charts, stacked bar charts, pie charts, and scatter plots are common examples.

You may automatically generate predefined charts, such as tracking the number of data points over time, or in a region. Or you may provide a more complex configuration interface, allowing the user to select which data fields map to the X and Y axes.

Divergent Features (Highlight Packages)

To promote uniqueness between your projects, there are a few extra feature packages you may wish to consider. These relate to different goals of the project and will have a large impact on how you implement the earlier features. This is not an exhaustive list; you may think of similar concepts that or combine aspects from more than one.

Highlight package A (Status management)

To better support public services having a way to view the status and report issues of amenities is an important way to crowd-source useful information.

- When viewing data points of amenities (i.e. bins, toilets, BBQs, etc…) users should be able to see the current status (i.e. operational, under maintenance, etc…)
- Users should be able to report an issue; doing so may change the status directly or may simply be a note on the amenity.
- Users should be able to view previously reported issues and any updates (i.e. fixes) that have happened in a simple timeline.

Highlight package B (Asset management)

You may consider creating an application that bridges the gap between display and reporting of data to the public and the backend infrastructure that allows an organisation to better manage the amenities and facilities they provide.

- Admin users should be the only ones able to add new data points or modify information about existing ones.
- Admin users may have the ability to assign different maintenance workers to issues; this may include planning routes/itineraries for a day.
- Admin users may track costs and export them as simple documents.

Highlight package C (Tourism/Suggested outings)

Tourists often don't know all the local sights when visiting a city, but many are interested in exploring the local culture. This is especially true for smaller walkable cities like Christchurch.

- Users may wish to focus on cultural, historical locations or artworks
- The application may suggest customized round-trip walking or biking tours that travel past selected landmarks or points of interest
- You may consider leaning towards a 'kiosk' application, i.e. set up in an airport or hotel for use by anonymous guests
- Users may want to also see shops and cafes around the area and plan trips that stop at these.

Highlight package D (Reviews and ratings)

Providing users the ability to review and rate locations, points of interest, amenities, etc… can expand the crowd-sourced data and allow other users to make more informed decisions.

- Users can add reviews and/or ratings to different locations. You may want to allow users to add more complex data types (i.e. images).
- Users can view past reviews and ratings to see what others think of a location
- Users can list (or otherwise see) the highest rating locations of different types easily

Highlight package E (Personal diary)

Taking a more individual approach, you may want to focus your application towards single users and give them functionality to record their own experiences.

- Users should be able to keep a diary of different locations, experiences, amenities they've visited.
- Users may be able to record extra types of information i.e. activity statistics for walking or cycling trips. This may include automatic calculation of burned calories based on distance etc…
- Users should be able to export their diaries or other information to share with others, such as overview image to use on social media or an email to send to friends and family

Datasets

Here are a number of datasets relating to Christchurch, you may prefer to look into data from other councils/regions.

- Drinking fountain locations https://opendata-christchurchcity.hub.arcgis.com/datasets/ChristchurchCity::drinking-fountain-opendata/explore
- Rubbish bin locations https://opendata-christchurchcity.hub.arcgis.com/datasets/7939c3c90abc4be188d4a836be099a8f_2/explore
- Park locations https://opendata-christchurchcity.hub.arcgis.com/datasets/8c5082665bbc4b1c84e5cb51efbe1d05_12/explore
- Nature area locations https://opendata-christchurchcity.hub.arcgis.com/datasets/81da7b7038784fd4b2e4d00372b245a1_8/explore
- Dog exercise area locations https://opendata-christchurchcity.hub.arcgis.com/datasets/e64abd50306c4417bf861025151fa18b_1/explore
- Cycleways https://opendata-christchurchcity.hub.arcgis.com/datasets/d9b5a1de32c4418a93b836fa254dfa8b_1/explore
- Walking and Mountain bike tracks https://opendata-christchurchcity.hub.arcgis.com/datasets/866f1329087d46b4b4bd4b370aea297f_6/explore
- Barbecue locations https://opendata-christchurchcity.hub.arcgis.com/datasets/2b3e3159b04d4fa1b483c9da86966677_3/explore
- Picnic table locations https://opendata-christchurchcity.hub.arcgis.com/datasets/34d4d5cd332f471288c02b09fe838be9_7/explore
- Public art installations https://opendata-christchurchcity.hub.arcgis.com/datasets/8c5d88842f3945b0b57890fff8f4f961_0/explore
- Plaque installations https://opendata-christchurchcity.hub.arcgis.com/datasets/653c5834b11447aab129649e638a6a35_9/explore